Energy-adaptive approximation

One binary carries several accuracy levels. The device picks one per deadline from the energy it has actually banked.

How it works

Offline, once. A benchmark is given an approximation plan controlled by a single integer knob. Bayesian optimisation runs each knob setting through the Fused simulator and records its output error and how many power cycles it costs. The useful settings become a **level table**. Because every level is the same program with a different knob value, all of them ship in one binary and switching costs nothing.

On the device, every deadline period. The device must deliver one result per period. It starts each task at the exact level and watches how much energy it has actually banked. If it is falling behind, it drops to a cheaper level and keeps the work already done; if energy recovers, it climbs back. **Nothing is predicted** — the controller reacts to measured progress.

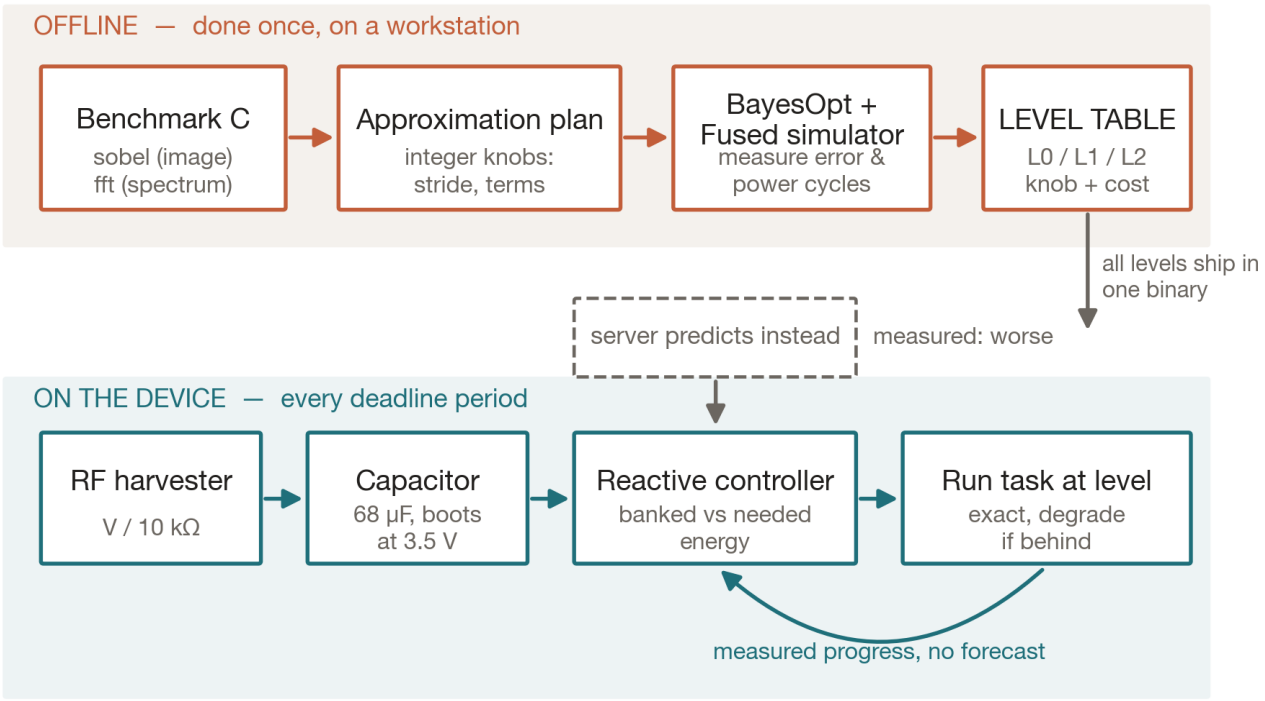


Figure 1. Levels are measured once offline; the choice between them is made on-device from banked energy.

The level tables we measured

Two benchmarks, each with one plan and a couple of integer knobs. In both, the exact level costs what the original costs, so the savings are honest.

sobel - image convolution. Knobs: the stride for scanning the min/max range, and the stride at which output pixels are computed (skipped pixels are filled from their block anchor).

level	knobs	output error	power cycles	energy / task	saving
L0	5, 1	0.000	45 / 62	10.77 mJ	27.4%
L1	**5, 2**	**0.019**	**13 / 62**	**3.11 mJ**	**79.0%**
L2	4, 4	0.280	8 / 62	1.91 mJ	87.1%

fft - spectrum. Knob: how many terms of the sin/cos Taylor series are kept, which sets the accuracy of every twiddle factor.

level	knobs	output error	power cycles	energy / task	saving
L0	7, 7	0.000	12 / 12	2.87 mJ	0%
L1	**4, 4**	**0.006**	**8 / 12**	**1.91 mJ**	**33.3%**
L2	2, 3	0.158	6 / 12	1.44 mJ	50.0%

What the system achieves

Policies were run over the energy traces through a power model validated against the Fused simulator (capacitor voltage within 5.2 mV, 9 of 9 power cycles matched). **Done** is the share of deadlines met; **error** is the mean output error of the results actually delivered; **wasted** is harvested energy that arrived with the capacitor already full. 480 runs in total.

The deadline decides whether adapting is worth anything

deadline	policy	done	error	wasted
8 s (tight)	always cheapest	85%	0.280	80 mJ
	reactive	64%	0.158	50 mJ
	oracle	85%	0.112	50 mJ
25 s	best fixed level	100%	0.019	116 mJ
	reactive	87%	0.023	78 mJ
	oracle	99%	0.013	78 mJ
40 s (loose)	best fixed level	100%	0.019	134 mJ
	reactive	**100%**	**0.008**	**70 mJ**
	oracle	100%	0.000	63 mJ

sobel on the measured RF traces. Reactive is highlighted where it wins outright.

> **The main result.** When the deadline is loose enough to leave a choice, reacting to banked energy beats the best fixed level on **both** counts: **2.4x less error and half the wasted energy**, meeting every deadline. On the measured solar trace the margin is wider still - 0.004 error against 0.019, and 58 mJ wasted against 150 mJ. When the deadline is tight, always running the cheapest level completes more tasks, at nearly twice the error.

The second benchmark agrees

On fft, reactive control is **identical to the oracle** at deadlines of 25 s and above - every deadline met at 0.000 error, against 0.006 for the best fixed level, while wasting less energy. fft is the easier case: its levels are cheap, so almost everything completes and the only question left is quality.

Why prediction lost to measurement

The original design predicted incoming energy, first with a moving average and then with a pretrained time-series model (Chronos-Bolt). We also built the server-in-the-loop version: the device reports its energy, a server predicts and picks the level, and the device reports the outcome back so the server can calibrate itself.

policy	done	error	wasted
reactive, on-device, no forecast	**100%**	**0.008**	**70 mJ**
server picks, device still corrects	100%	0.077	116 mJ
server picks alone, learns from outcomes	100%	0.108	117 mJ
moving-average forecast	100%	0.049	98 mJ

Every predictive variant lands above the device that simply measures its own progress. The radio is not the reason - at 0 uJ and 200 uJ per round trip the numbers are the same, so the round trip is affordable and the decision quality is what limits it. Feedback barely helped either, because calibration fixes bias but not resolution: the server ends up polarised between exact and cheapest and rarely picks the middle level at all.

Honest limits

1. Tight deadlines still favour a fixed level. At 8 s, always running the cheapest level completes 85% against reactive's 64%. Adapting pays when there is slack to spend, not when there is none.

2. The level tables come from hand-written code. They are an upper bound on what these transformations can deliver, not a result about automatically generated plans.

3. Two benchmarks, and they are the two that work. A classifier (ar-iclib) could not be given a usable dial at all: its output is a discrete two-class decision, so sampling its model flips decisions unpredictably - intermediate strides gave 0.62-1.0 error where the coarsest gave 0.398. Continuous outputs degrade smoothly; discrete decisions fall off a cliff.

What would strengthen it next

priority	action	why it matters
1	Longer energy traces	the RF traces are 25-52 s; only the 300 s solar trace has many decision point
2	A benchmark that is not hand-tuned	shows the dial can be produced automatically, not just measured
3	Close the tight-deadline gap	reactive starts a period optimistically and cannot always recover